

By default, `dweet.io` is a public service, so any app can publish or subscribe to messages. When publishing messages, ***you'll want to specify a unique name for your device.*** We used `'temperature-simulator-deitel-python'` (line 17).¹ Lines 18–21 define a Python dictionary, which will store the current sensor information. Dweepy will convert this into JSON when it sends the dweet.

¹To truly guarantee a unique name, `dweet.io` can create one for you. The Dweepy documentation explains how to do this.

[lick here to view code image](#)

```
1 # simulator.py
2 """A connected thermostat simulator that publishes JSON
3 messages to dweet.io"""
4 import dweepy
5 import sys
6 import time
7 import random
8
9 MIN_CELSIUS_TEMP = -25
10 MAX_CELSIUS_TEMP = 45
11 MAX_TEMP_CHANGE = 2
12
13 # get the number of messages to simulate and delay between them
14 NUMBER_OF_MESSAGES = int(sys.argv[1])
15 MESSAGE_DELAY = int(sys.argv[2])
16
17 dweeter = 'temperature-simulator-deitel-python' # provide a unique name
18 thermostat = {'Location': 'Boston, MA, USA',
19               'Temperature': 20,
20               'LowTempWarning': False,
21               'HighTempWarning': False}
22
```

Lines 25–53 produce the number of simulated message you specify. During each iteration of the loop, we

- generate a random temperature change in the range -2 to $+2$ degrees and modify the temperature,
- ensure that the temperature remains in the allowed range,
- check whether the low- or high-temperature sensor has been triggered and update the thermostat dictionary accordingly,
- display how many messages have been generated so far,
- use Dweepy to send the message to `dweet.io` (line 52), and
- use the `time` module's `sleep` function to wait the specified amount of time before generating another message.

[lick here to view code image](#)

```
23 print('Temperature simulator starting')
24
```

```

25 for message in range(NUMBER_OF_MESSAGES):
26     # generate a random number in the range  -MAX_TEMP_CHANGE
27     # through MAX_TEMP_CHANGE and add it to the current temperature
28     thermostat['Temperature'] += random.randrange(
29         -MAX_TEMP_CHANGE, MAX_TEMP_CHANGE + 1)
30
31     # ensure that the temperature stays within range
32     if thermostat['Temperature'] < MIN_CELSIUS_TEMP:
33         thermostat['Temperature'] = MIN_CELSIUS_TEMP
34
35     if thermostat['Temperature'] > MAX_CELSIUS_TEMP:
36         thermostat['Temperature'] = MAX_CELSIUS_TEMP
37
38     # check for low temperature warning
39     if thermostat['Temperature'] < 3:
40         thermostat['LowTempWarning'] = True
41     else:
42         thermostat['LowTempWarning'] = False
43
44     # check for high temperature warning
45     if thermostat['Temperature'] > 35:
46         thermostat['HighTempWarning'] = True
47     else:
48         thermostat['HighTempWarning'] = False
49
50     # send the dweet to dweet.io via dweeepy
51     print(f'Messages sent: {message + 1}\r', end='')
52     dweeepy.dweet_for(dweeter, thermostat)
53     time.sleep(MESSAGE_DELAY)
54
55 print('Temperature simulator finished')

```

You do not need to register to use the service. On the first call to dweeepy's **dweet_for** function to send a dweet (line 52), `dweet.io` creates the device name. The function receives as arguments the device name (`dweeter`) and a dictionary representing the message to send (`thermostat`). Once you execute the script, you can immediately begin tracking the messages on the `dweet.io` site by going to the following address in your web browser:

```
https://dweet.io/follow/temperature-simulator-deitel-python
```

If you use a different device name, replace "temperature-simulator-deitel-python" with the name you used. The web page contains two tabs. The **Visual** tab shows you the individual data items, displaying a sparkline for any numerical values. The **Raw** tab shows you the actual JSON messages that Dweeepy sent to `dweet.io`.

16.8.4 Creating the Dashboard with Freeboard.io

The sites `dweet.io` and `freeboard.io` are run by the same company. In the `dweet.io` webpage discussed in the preceding section, you can click the **Create a Custom Dashboard** button to open a new browser tab, with a default dashboard already implemented for the temperature sensor. By default, `freeboard.io` will configure a data source named `Dweet` and auto-generate a dashboard containing one pane for each value in the dweet JSON. Within each pane, a text widget will display the corresponding value as the messages arrive.

If you prefer to create your own dashboard, you can use the steps in section 16.8.2 to create a data source (this time selecting `Dweeepy`) and create new panes and widgets, or you can you

modify the auto-generated dashboard.

Below are three screen captures of a dashboard consisting of four widgets:

- A **Gauge** widget showing the current temperature. For this widget's **VALUE** setting, we selected the data source's `Temperature` field. We also set the **UNITS** to `Celsius` and the **MINIMUM** and **MAXIMUM** values to `-25` and `45` degrees, respectively.
- A **Text** widget to show the current temperature in Fahrenheit. For this widget, we set the **INCLUDE SPARKLINE** and **ANIMATE VALUE CHANGES** to **YES**. For this widget's **VALUE** setting, we again selected the data source's `Temperature` field, then added to the end of the **VALUE** field

```
* 9 / 5 + 32
```

to perform a calculation that converts the Celsius temperature to Fahrenheit. We also specified `Fahrenheit` in the **UNITS** field.

- Finally, we added two **Indicator Light** widgets. For the first **Indicator Light**'s **VALUE** setting, we selected the data source's `LowTempWarning` field, set the **TITLE** to `Freeze Warning` and set the **ON TEXT** value to `LOW TEMPERATURE WARNING—ON TEXT` indicates the text to display when value is `true`. For the second **Indicator Light**'s **VALUE** setting, we selected the data source's `HighTempWarning` field, set the **TITLE** to `High Temperature Warning` and set the **ON TEXT** value to `HIGH TEMPERATURE WARNING`.



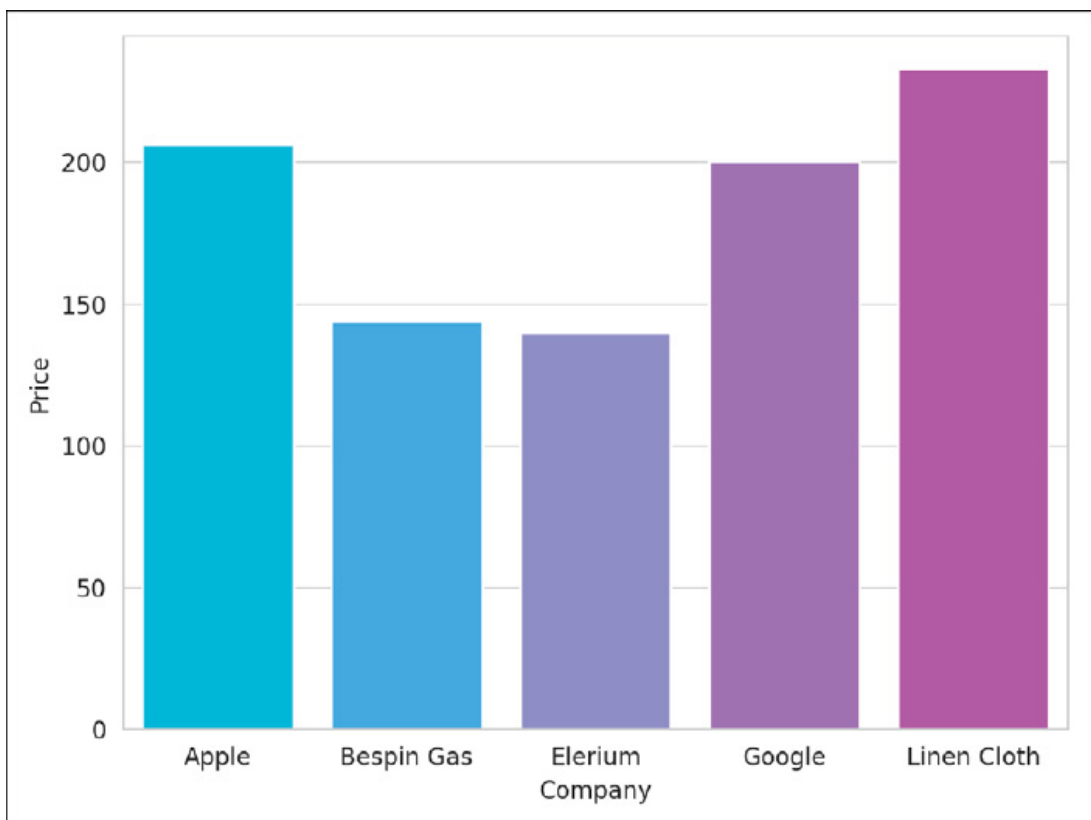
16.8.5 Creating a Python PubNub Subscriber

PubNub provides the `pubnub` Python module for conveniently performing pub/sub operations. They also provide seven sample streams for you to experiment with—four real-time streams and three simulated streams: ²

² <https://www.pubnub.com/developers/realtime-data-streams/>.

- Twitter Stream—provides up to 50 tweets-per-second from the Twitter live stream and does not require your Twitter credentials.
- Hacker News Articles—this site’s recent articles.
- State Capital Weather—provides weather data for the U.S. state capitals.
- Wikipedia Changes—a stream of Wikipedia edits.
- Game State Sync—simulated data from a multiplayer game.
- Sensor Network—simulated data from radiation, humidity, temperature and ambient light sensors.
- Market Orders—simulated stock orders for five companies.

In this section, you’ll use the **pubnub module** to subscribe to their simulated Market Orders stream, then visualize the changing stock prices as a Seaborn barplot, like:



Of course, you also can publish messages to streams. For details, see the pubnub module’s documentation at <https://www.pubnub.com/docs/python/pubnub-python-sdk>.

To prepare for using PubNub in Python, execute the following command to install the latest version of the pubnub module—the `'>=4.1.2'` ensures that at a minimum the 4.1.2 version of the pubnub module will be installed:

```
pip install "pubnub>=4.1.2"
```

The script `stocklistener.py` that subscribes to the stream and visualizes the stock prices is defined in the `ch16` folder’s `pubnub` subfolder. We break the script into pieces here for discussion purposes.

Message Format

The simulated Market Orders stream returns JSON objects containing five key–value pairs with the keys 'bid_price', 'order_quantity', 'symbol', 'timestamp' and 'trade_type'. For this example, we'll use only the 'bid_price' and 'symbol'. The PubNub client returns the JSON data to you as a Python dictionary.

Importing the Libraries

Lines 3–13 import the libraries used in this example. We discuss the PubNub types imported in lines 10–13 as we encounter them below.

[lick here to view code image](#)

```
1 # stocklistener.py
2 """Visualizing a PubNub live stream."""
3 from matplotlib import animation
4 import matplotlib.pyplot as plt
5 import pandas as pd
6 import random
7 import seaborn as sns
8 import sys
9
10 from pubnub.callbacks import SubscribeCallback
11 from pubnub.enums import PNStatusCategory
12 from pubnub.pnconfiguration import PNConfiguration
13 from pubnub.pubnub import PubNub
14
```

List and DataFrame Used for Storing Company Names and Prices

The list `companies` contains the names of the companies reported in the Market Orders stream, and the pandas DataFrame `companies_df` is where we'll store each company's last price. We'll use this DataFrame with Seaborn to display a bar chart.

[lick here to view code image](#)

```
15 companies = ['Apple', 'Bespun Gas', 'Elerium', 'Google', 'Linen Cloth']
16
17 # DataFrame to store last stock prices
18 companies_df = pd.DataFrame(
19     {'company': companies, 'price': [0, 0, 0, 0, 0]}
20 )
```

Class `SensorSubscriberCallback`

When you subscribe to a PubNub stream, you must add a listener that receives status notifications and messages from the channel. This is similar to the Tweepy listeners you've defined previously. To create your listener, you must define a subclass of `SubscribeCallback` (module `pubnub.callbacks`), which we discuss after the code:

[lick here to view code image](#)

```
21 class SensorSubscriberCallback(SubscribeCallback):
22     """SensorSubscriberCallback receives messages from PubNub."""
23     def __init__(self, df, limit=1000):
```

```

24     """Create instance variables for tracking number of tweets."""
25     self.df = df # DataFrame to store last stock prices
26     self.order_count = 0
27     self.MAX_ORDERS = limit # 1000 by default
28     super().__init__() # call superclass's init
29
30     def status(self, pubnub, status):
31         if status.category == PNStatusCategory.PNConnectedCategory:
32             print('Connected to PubNub')
33         elif status.category == PNStatusCategory.PNAcknowledgmentCategory:
34             print('Disconnected from PubNub')
35
36     def message(self, pubnub, message):
37         symbol = message.message['symbol']
38         bid_price = message.message['bid_price']
39         print(symbol, bid_price)
40         self.df.at[companies.index(symbol), 'price'] = bid_price
41         self.order_count += 1
42
43         # if MAX_ORDERS is reached, unsubscribe from PubNub channel
44         if self.order_count == self.MAX_ORDERS:
45             pubnub.unsubscribe_all()
46

```

Class `SensorSubscriberCallback`'s `__init__` method stores the `DataFrame` in which each new stock price will be placed. The PubNub client calls overridden method `status` each time a new status message arrives. In this case, we're checking for the notifications that indicate that we've subscribed to or unsubscribed from a channel.

The PubNub client calls overridden method `message` (lines 36–45) when a new message arrives from the channel. Lines 37 and 38 get the company name and price from the message, which we print so you can see that messages are arriving. Line 40 uses the `DataFrame` method `at` to locate the appropriate company's row and its 'price' column, then assign that element the new price. Once the `order_count` reaches `MAX_ORDERS`, line 45 calls the PubNub client's `unsubscribe_all` method to unsubscribe from the channel.

Function Update

This example visualizes the stock prices using the animation techniques you learned in chapter 6's Intro to Data Science section. Function `update` specifies how to draw one animation frame and is called repeatedly by the `FuncAnimation` we'll define shortly. We use Seaborn function `barplot` to visualize data from the `companies_df` `DataFrame`, using its 'company' column values on the x-axis and 'price' column values on the y-axis.

[lick here to view code image](#)

```

47 def update(frame_number):
48     """Configures bar plot contents for each animation frame."""
49     plt.cla() # clear old barplot
50     axes = sns.barplot(
51         data=companies_df, x='company', y='price', palette='cool')
52     axes.set(xlabel='Company', ylabel='Price')
53     plt.tight_layout()
54

```

Configuring the Figure

In the main part of the script, we begin by setting the Seaborn plot style and creating the Figure object in which the barplot will be displayed:

[lick here to view code image](#)

```
55 if __name__ == '__main__':
56     sns.set_style('whitegrid') # white background with gray grid lines
57     figure = plt.figure('Stock Prices') # Figure for animation
58
```

Configuring the FuncAnimation and Displaying the Window

Next, we set up the FuncAnimation that calls function update, then call Matplotlib's show method to display the Figure. Normally, this method blocks the script from continuing until you close the Figure. Here, we pass the block=False keyword argument to allow the script to continue so we can configure the PubNub client and subscribe to a channel.

[lick here to view code image](#)

```
59 # configure and start animation that calls function update
60 stock_animation = animation.FuncAnimation(
61     figure, update, repeat=False, interval=33)
62 plt.show(block=False) # display window
63
```

Configuring the PubNub Client

Next, we configure the PubNub subscription key, which the PubNub client uses in combination with the channel name to subscribe to the channel. The key is specified as an attribute of the PNConfiguration object (module pubnub.pnconfiguration), which line 69 passes to the new PubNub client object (module pubnub.pubnub). Lines 70–72 create the SensorSubscriberCallback object and pass it to the PubNub client's add_listener method to register it to receive messages from the channel. We use a command-line argument to specify the total number of messages to process.

[lick here to view code image](#)

```
64 # set up pubnub-market-orders sensor stream key
65 config = PNConfiguration()
66 config.subscribe_key = 'sub-c-4377ab04-f100-11e3-bffd-02ee2ddab7fe'
67
68 # create PubNub client and register a SubscribeCallback
69 pubnub = PubNub(config)
70 pubnub.add_listener(
71     SensorSubscriberCallback(df=companies_df,
72                             limit=int(sys.argv[1] if len(sys.argv) > 1 else 1000))
73
```

Subscribing to the Channel

The following statement completes the subscription process, indicating that we wish to receive messages from the channel named 'pubnub-market-orders'. The execute method starts the stream.

[lick here to view code image](#)

```
74     # subscribe to pubnub-sensor-network channel and begin streaming
75     pubnub.subscribe().channels('pubnub-market-orders').execute()
76
```

Ensuring the Figure Remains on the Screen

The second call to Matplotlib's `show` method ensures that the `Figure` remains on the screen until you close its window.

[lick here to view code image](#)

```
77     plt.show()     # keeps graph on screen until you dismiss its window
```

16.9 WRAP-UP

In this chapter, we introduced big data, discussed how large data is getting and discussed hardware and software infrastructure for working with big data. We introduced traditional relational databases and Structured Query Language (SQL) and used the `sqlite3` module to create and manipulate a `books` database in SQLite. We also demonstrated loading SQL query results into pandas `DataFrames`.

We discussed the four major types of NoSQL databases—key–value, document, columnar and graph—and introduced NewSQL databases. We stored JSON tweet objects as documents in a cloud-based MongoDB Atlas cluster, then summarized them in an interactive visualization displayed on a Folium map.

We introduced Hadoop and how it's used in big-data applications. You configured a multi-node Hadoop cluster using the Microsoft Azure HDInsight service, then created and executed a Hadoop MapReduce task using Hadoop streaming.

We discussed Spark and how it's used in high-performance, real-time big-data applications. You used Spark's functional-style filter/map/reduce capabilities, first on a Jupyter Docker stack that runs locally on your own computer, then again using a Microsoft Azure HDInsight multi-node Spark cluster. Next, we introduced Spark streaming for processing data in mini-batches. As part of that example, we used Spark SQL to query data stored in Spark `DataFrames`.

The chapter concluded with an introduction to the Internet of Things (IoT) and the publish/subscribe model. You used Freeboard.io to create a dashboard visualization of a live sample stream from PubNub. You simulated an Internet-connected thermostat which published messages to the free `dweet.io` service using the Python module `Dweeepy`, then used Freeboard.io to visualize the simulated device's data. Finally, you subscribed to a PubNub sample live stream using their Python module.

Thanks for reading *Python for Programmers*. We hope that you enjoyed the book and that you found it entertaining and informative. Most of all we hope you feel empowered to apply the technologies you've learned to the challenges you'll face in your career.



Photo courtesy of Paul Deitel

Expert-Led Video Training on Python, Data Science, AI, Big Data and the Cloud

From **PAUL DEITEL**

Python Fundamentals LiveLessons integrates coverage of the core Python language, Python Standard Library and key data science libraries with a range of data science and artificial intelligence case studies, including:

- Natural language processing
- Data mining Twitter® for sentiment analysis
- IBM® Watson™ and cognitive computing
- Machine learning with classification, regression and clustering
- Deep learning with convolutional and recurrent neural networks
- Big data: Hadoop®, Spark™ and NoSQL
- Internet of Things (IoT)

You'll receive a **hands-on** introduction to Python and data science with world-renowned trainer Paul Deitel as he presents 500+ real-world examples, including 40 larger scripts and case studies. Quickly master the latest Python coding idioms using the interactive **IPython interpreter** with code in **Jupyter Notebooks**, and benefit from additional insights and programming-language comparisons based on Paul's 38 years of programming and teaching experience in Java, C, C++, C++, Swift, iOS, Android and Internet and web programming.

Save 40%* —Use coupon code VIDEO40
informit.com/Deitel

*Discount code VIDEO40 confers a 40% discount off the list price of featured video when purchased on InformIT. Offer is subject to change.



Photo by izusek/gettyimages

Register Your Product at informit.com/register

Access additional benefits and **save 35%** on your next purchase

- Automatically receive a coupon for 35% off your next purchase, valid for 30 days. Look for your code in your InformIT cart or the Manage Codes section of your account page.
- Download available product updates.
- Access bonus material if available.*
- Check the box to hear from us and receive exclusive offers on new editions and related products.

**Registration benefits vary by product. Benefits will be listed on your account page under Registered Products.*

InformIT.com—The Trusted Technology Learning Source

InformIT is the online home of information technology brands at Pearson, the world's foremost education company. At InformIT.com, you can:

- Shop our books, eBooks, software, and video training
- Take advantage of our special offers and promotions (informit.com/promotions)
- Sign up for special offers and content newsletter (informit.com/newsletters)
- Access thousands of free chapters and video lessons

Connect with InformIT—Visit informit.com/community



informIT[®]
the trusted technology learning source

Addison-Wesley • Adobe Press • Cisco Press • Microsoft Press • Pearson IT Certification • Prentice Hall • Que • Sams • Peachpit Press